

Chapter 3

JavaScript Operators & Expressions

This chapter is the foundation for **calculations, comparisons, conditions, and decision-making** in JavaScript.

3.1 Learning Objectives

After completing this chapter, students should be able to:

- Define an operator and operand.
 - Explain different types of JavaScript operators.
 - Use arithmetic operators.
 - Use assignment operators.
 - Use comparison operators.
 - Use logical operators.
 - Use increment and decrement operators.
 - Use unary operators.
 - Use the ternary operator.
 - Understand string concatenation.
 - Understand operator precedence and associativity.
 - Explain expressions.
 - Understand type coercion during operations.
 - Write practical programs using operators.
-

3.2 What is an Operator?

An **operator** is a symbol or keyword that performs an operation on one or more values.

Example:

```
let sum = 10 + 20;
```

Here:

10 → Operand

+ → Operator

20 → Operand

The + operator performs addition.

3.3 What is an Operand?

An **operand** is a value on which an operator performs an operation.

Example:

let result = 10 + 5;

10 → Operand

+ → Operator

5 → Operand

3.4 Types of JavaScript Operators

The major categories are:

JavaScript Operators

- |
- |— Arithmetic
- |— Assignment
- |— Comparison
- |— Logical
- |— Increment / Decrement
- |— Unary
- |— Conditional / Ternary
- |— String

- └— Bitwise
- └— Nullish Coalescing
- └— Optional Chaining

For this chapter, concentrate first on the operators most commonly used in application programming.

3.5 Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator Meaning

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder
**	Exponentiation

3.5.1 Addition +

```
let a = 10;
```

```
let b = 20;
```

```
let result = a + b;
```

```
console.log(result);
```

Output:

30

3.5.2 Subtraction -

let a = 20;

let b = 10;

console.log(a - b);

Output:

10

3.5.3 Multiplication *

let a = 10;

let b = 5;

console.log(a * b);

Output:

50

3.5.4 Division /

let a = 20;

let b = 5;

console.log(a / b);

Output:

4

JavaScript's number type uses floating-point representation, so:

console.log(5 / 2);

Output:

2.5

3.5.5 Remainder %

The % operator returns the remainder after division.

```
console.log(10 % 3);
```

Output:

1

Because:

$10 \div 3$

Quotient = 3

Remainder = 1

Important application

Checking whether a number is even:

```
let number = 20;
```

```
console.log(number % 2);
```

Output:

0

Therefore:

`number % 2 === 0`

can be used to test whether a number is even.

3.5.6 Exponentiation **

Used to calculate powers.

```
console.log(2 ** 3);
```

Output:

8

Because:

$$2^3 = 8$$

Another example:

```
let square = 5 ** 2;
```

```
console.log(square);
```

Output:

25

3.6 Assignment Operators

Assignment operators assign values to variables.

The basic assignment operator is:

=

Example:

```
let x = 10;
```

3.6.1 Addition Assignment +=

```
let x = 10;
```

```
x += 5;
```

```
console.log(x);
```

Output:

15

It is equivalent to:

```
x = x + 5;
```

3.6.2 Subtraction Assignment -=

```
let x = 10;
```

```
x -= 3;
```

```
console.log(x);
```

Output:

7

Equivalent to:

```
x = x - 3;
```

3.6.3 Multiplication Assignment *=

```
let x = 10;
```

```
x *= 2;
```

```
console.log(x);
```

Output:

20

3.6.4 Division Assignment /=

```
let x = 20;
```

```
x /= 4;
```

```
console.log(x);
```

Output:

5

3.6.5 Remainder Assignment %=

```
let x = 10;
```

```
x %= 3;
```

```
console.log(x);
```

Output:

1

3.6.6 Exponentiation Assignment **=

```
let x = 2;
```

```
x **= 3;
```

```
console.log(x);
```

Output:

8

3.7 Comparison Operators

Comparison operators compare two values.

The result is generally:

true

or:

false

Operator Meaning

== Loose equality

=== Strict equality

!= Loose inequality

!== Strict inequality

> Greater than

< Less than

>= Greater than or equal

<= Less than or equal

3.8 Equal ==

The == operator performs loose equality comparison and may perform type coercion.

```
console.log(10 == "10");
```

Output:

true

Although one value is a number and the other is a string, coercion occurs.

3.9 Strict Equal ===

The === operator checks both:

1. Value

2. Type

Example:

```
console.log(10 === "10");
```

Output:

false

Because:

10 → number

"10" → string

3.10 Not Equal !=

```
console.log(10 != 5);
```

Output:

true

3.11 Strict Not Equal !==

```
console.log(10 !== "10");
```

Output:

true

Because the types are different.

3.12 Greater Than >

```
console.log(20 > 10);
```

Output:

true

3.13 Less Than <

```
console.log(10 < 20);
```

Output:

true

3.14 Greater Than or Equal >=

```
console.log(20 >= 20);
```

Output:

true

3.15 Less Than or Equal <=

```
console.log(15 <= 20);
```

Output:

true

3.16 Logical Operators

Logical operators are used to combine or manipulate Boolean conditions.

The three fundamental logical operators are:

Operator Meaning

&& AND

,

! NOT

3.17 Logical AND &&

AND returns a truthy result only when both operands/conditions are truthy.

For Boolean values:

```
console.log(true && true);
```

Output:

true

But:

```
console.log(true && false);
```

Output:

false

Truth Table

A B A && B

T T T

T F F

F T F

F F F

Real-life example

A student can enter an exam if:

Attendance ≥ 75

AND

Exam Fee Paid = true

JavaScript:

```
let attendance = 80;
```

```
let feePaid = true;
```

```
console.log(attendance  $\geq 75$  && feePaid === true);
```

Output:

true

3.18 Logical OR ||

OR returns a truthy result when at least one operand/condition is truthy.

```
console.log(true || false);
```

Output:

true

Truth Table

A B A || B

T T T

T F T

F T T

F F F

Example:

```
let isAdmin = false;
```

```
let isTeacher = true;
```

```
console.log(isAdmin || isTeacher);
```

Output:

true

3.19 Logical NOT !

NOT reverses a Boolean value.

```
console.log(!true);
```

Output:

false

And:

```
console.log(!false);
```

Output:

true

Example:

```
let loggedIn = false;
```

```
console.log(!loggedIn);
```

Output:

true

3.20 Increment Operator ++

Increment increases a value by 1.

```
let x = 10;
```

```
x++;
```

```
console.log(x);
```

Output:

11

Equivalent to:

```
x = x + 1;
```

3.21 Decrement Operator --

Decrement decreases a value by 1.

```
let x = 10;
```

```
x--;
```

```
console.log(x);
```

Output:

9

Equivalent to:

```
x = x - 1;
```

3.22 Prefix and Postfix

This is an important MCA/viva topic.

Postfix Increment

```
let x = 10;
```

```
let y = x++;
```

```
console.log(x);
```

```
console.log(y);
```

Output:

11

10

Why?

The old value is used first, then x is incremented.

Prefix Increment

```
let x = 10;
```

```
let y = ++x;
```

```
console.log(x);
```

```
console.log(y);
```

Output:

11

11

Here x is incremented first and then its new value is used.

3.23 Prefix vs Postfix

Remember:

$x++ \rightarrow$ use first, increase later

$++x \rightarrow$ increase first, use later

Similarly:

$x-- \rightarrow$ use first, decrease later

$--x \rightarrow$ decrease first, use later

3.24 Unary Operators

A unary operator operates on one operand.

Example:

let x = 10;

```
console.log(-x);
```

Output:

-10

Another example:

```
let value = "100";
```

```
console.log(+value);
```

Output:

100

The unary + can convert certain values to numbers.

3.25 Ternary Operator

The ternary operator is a compact alternative to a simple if...else.

Syntax:

condition ? valueIfTrue : valueIfFalse

Example:

```
let age = 20;
```

```
let result = age >= 18 ? "Adult" : "Minor";
```

```
console.log(result);
```

Output:

Adult

Explanation:

age >= 18

↓

true

↓

"Adult"

3.26 Practical Example — Pass or Fail

```
let marks = 65;
```

```
let result = marks >= 40 ? "Pass" : "Fail";
```

```
console.log(result);
```

Output:

Pass

3.27 Practical Example — Even or Odd

```
let number = 17;
```

```
let result = number % 2 === 0 ? "Even" : "Odd";
```

```
console.log(result);
```

Output:

Odd

3.28 String Concatenation

The + operator can also combine strings.

```
let firstName = "Rahul";
```

```
let lastName = "Kumar";
```

```
let fullName = firstName + " " + lastName;
```

```
console.log(fullName);
```

Output:

Rahul Kumar

This is called **string concatenation**.

3.29 String Concatenation with Numbers

Consider:

```
console.log("Age: " + 22);
```

Output:

Age: 22

JavaScript converts the number to a string for concatenation.

3.30 Important Output Question

What is the output?

```
console.log(10 + 20);
```

```
console.log("10" + 20);
```

```
console.log(10 + "20");
```

```
console.log("10" + "20");
```

Answer:

30

1020

1020

1020

Why?

When + involves a string, it can perform string concatenation.

3.31 Other Arithmetic Type Coercion

Consider:

```
console.log("10" - 2);
```

```
console.log("10" * 2);
```

```
console.log("10" / 2);
```

Output:

8

20

5

The string "10" is converted to a number for these operations.

3.32 Operator Precedence

When an expression contains multiple operators, JavaScript follows precedence rules.

Example:

```
let result = 10 + 5 * 2;
```

```
console.log(result);
```

Output:

20

Not:

30

Why?

Multiplication has higher precedence than addition.

Therefore:

$$10 + 5 \times 2$$
$$= 10 + 10$$

= 20

3.33 Parentheses

Parentheses can change the order of evaluation.

```
let result = (10 + 5) * 2;
```

```
console.log(result);
```

Output:

30

Because:

$$(10 + 5) \times 2$$
$$= 15 \times 2$$
$$= 30$$

3.34 Basic Precedence Order

For the concepts covered so far, students can remember approximately:

1. Parentheses
2. Exponentiation
3. Unary operators / increment-decrement
4. Multiplication, division, remainder
5. Addition, subtraction
6. Comparison
7. Logical AND
8. Logical OR
9. Ternary
10. Assignment

When unsure, **use parentheses**.

3.35 What is an Expression?

An **expression** is a combination of values, variables, operators, and/or function calls that produces a value.

Example:

$10 + 20$

produces:

30

Therefore it is an expression.

Another:

$\text{age} \geq 18$

produces:

true

3.36 Examples of Expressions

Arithmetic expression

$a + b$

Comparison expression

$\text{age} \geq 18$

Logical expression

$\text{age} \geq 18 \ \&\& \ \text{hasID}$

Assignment expression

$x = 10$

Ternary expression

$\text{age} \geq 18 \ ? \ \text{"Adult"} : \text{"Minor"}$

Practical 3.1 — Arithmetic Calculator

let a = 20;

let b = 10;

console.log("Addition:", a + b);

console.log("Subtraction:", a - b);

console.log("Multiplication:", a * b);

console.log("Division:", a / b);

console.log("Remainder:", a % b);

console.log("Power:", a ** 2);

Practical 3.2 — Student Result

let java = 80;

let dbms = 75;

let web = 90;

let total = java + dbms + web;

let average = total / 3;

console.log("Total:", total);

console.log("Average:", average);

Practical 3.3 — Assignment Operators

let x = 10;

console.log(x);

```
x += 5;  
console.log(x);
```

```
x -= 2;  
console.log(x);
```

```
x *= 3;  
console.log(x);
```

```
x /= 2;  
console.log(x);
```

Ask students to predict every value.

Practical 3.4 — Comparison

```
let a = 20;
```

```
let b = 10;
```

```
console.log(a > b);
```

```
console.log(a < b);
```

```
console.log(a >= b);
```

```
console.log(a <= b);
```

```
console.log(a == b);
```

```
console.log(a != b);
```

Practical 3.5 — Strict Comparison

```
let a = 10;  
let b = "10";
```

```
console.log(a == b);  
console.log(a === b);  
console.log(a != b);  
console.log(a !== b);
```

Ask students:

Why are the results different?

Practical 3.6 — Logical Operators

```
let age = 22;  
let hasID = true;
```

```
console.log(age >= 18 && hasID);  
console.log(age >= 18 || hasID);  
console.log(!hasID);
```

Practical 3.7 — Eligibility

```
let age = 21;  
let qualification = true;
```

```
let eligible = age >= 18 && qualification;
```

```
console.log("Eligible:", eligible);
```

Practical 3.8 — Increment/Decrement

let x = 10;

console.log(x);

x++;

console.log(x);

x--;

console.log(x);

Practical 3.9 — Prefix/Postfix

let a = 10;

console.log(a++);

console.log(a);

Then:

let b = 10;

console.log(++b);

console.log(b);

Ask students to explain the difference.

Practical 3.10 — Ternary Operator

```
let marks = 70;
```

```
let result = marks >= 40 ? "Pass" : "Fail";
```

```
console.log(result);
```

Practical 3.11 — Even/Odd

```
let number = 25;
```

```
let result = number % 2 === 0 ? "Even" : "Odd";
```

```
console.log(result);
```

Practical 3.12 — Largest of Two Numbers

```
let a = 50;
```

```
let b = 70;
```

```
let largest = a > b ? a : b;
```

```
console.log("Largest:", largest);
```

Practical 3.13 — Largest of Three Numbers

Using nested ternary:

```
let a = 50;
```

```
let b = 70;
```

```
let c = 40;
```

```
let largest = a > b
```

```
  ? (a > c ? a : c)
```

```
  : (b > c ? b : c);
```

```
console.log("Largest:", largest);
```

Tell students that although this demonstrates the ternary operator, a normal if...else structure is often easier to read for more complicated logic.

Practical 3.14 — Simple Calculator

```
let a = 20;
```

```
let b = 5;
```

```
let operator = "+";
```

```
let result;
```

```
if (operator === "+") {
```

```
  result = a + b;
```

```
} else if (operator === "-") {
```

```
  result = a - b;
```

```
} else if (operator === "*") {
```

```
  result = a * b;
```

```
} else if (operator === "/") {
```

```
  result = a / b;
```

```
} else {
```

```
  result = "Invalid operator";
```

```
}
```

```
console.log("Result:", result);
```

This program prepares students for the next chapter on **conditional statements**.

Practical 3.15 — Electricity Bill Concept

Suppose:

Units = 100

Rate = ₹5

Program:

```
let units = 100;
```

```
let rate = 5;
```

```
let bill = units * rate;
```

```
console.log("Electricity Bill:", bill);
```

Later, conditions can be added for different slabs.

Practical 3.16 — Shopping Bill

```
let price = 500;
```

```
let quantity = 3;
```

```
let subtotal = price * quantity;
```

```
let discount = 50;
```

```
let finalAmount = subtotal - discount;
```

```
console.log("Subtotal:", subtotal);  
console.log("Discount:", discount);  
console.log("Final Amount:", finalAmount);
```

Practical 3.17 — Temperature Conversion

```
let celsius = 30;  
  
let fahrenheit = (celsius * 9 / 5) + 32;  
  
console.log("Celsius:", celsius);  
console.log("Fahrenheit:", fahrenheit);
```

Practical 3.18 — BMI Calculation

Formula:

$\text{BMI} = \text{weight} / (\text{height} \times \text{height})$

Example:

```
let weight = 70;  
let height = 1.75;  
  
let bmi = weight / (height * height);
```

```
console.log("BMI:", bmi);
```

The classification logic can be taught later with conditional statements.

OUTPUT-PREDICTION PRACTICE

These are excellent for an MCA classroom.

Question 1

let x = 10;

let y = 20;

console.log(x + y);

Answer:

30

Question 2

console.log(10 + "20");

Answer:

1020

Question 3

console.log("10" - 5);

Answer:

5

Question 4

let x = 10;

console.log(x++);

console.log(x);

Answer:

10

11

Question 5

let x = 10;

console.log(++x);

console.log(x);

Answer:

11

11

Question 6

console.log(5 == "5");

console.log(5 === "5");

Answer:

true

false

Question 7

console.log(10 > 5 && 20 > 15);

Answer:

true

Question 8

console.log(10 > 20 || 5 < 10);

Answer:

true

Question 9

```
console.log(!(10 > 5));
```

Answer:

false

Question 10

```
console.log(10 + 5 * 2);
```

Answer:

20

CHAPTER 3 ASSIGNMENTS

Assignment 1 — Basic Calculator

Create a program that accepts two numbers and displays:

- Addition
 - Subtraction
 - Multiplication
 - Division
 - Remainder
 - Power
-

Assignment 2 — Student Marks

Create a program for five subjects and calculate:

- Total
- Average

- Percentage

Assignment 3 — Salary

Given:

Basic Salary

HRA

DA

calculate gross salary.

Assignment 4 — Shopping

Given:

Product price

Quantity

Discount

calculate the final amount.

Assignment 5 — Even/Odd

Use % and the ternary operator to determine whether a number is even or odd.

Assignment 6 — Positive/Negative

Use the ternary operator to determine whether a number is positive or negative.

Assignment 7 — Largest Number

Find the largest among two numbers.

Then try three numbers.

Assignment 8 — Eligibility

Create a program that checks whether a person is eligible based on:

Age ≥ 18

AND

Has valid ID

Viva Questions

1. What is an operator?
2. What is an operand?
3. What is an arithmetic operator?
4. What is the % operator?
5. What is the difference between % and /?
6. What is the purpose of **?
7. What is an assignment operator?
8. What does += mean?
9. Difference between == and ===?
10. Difference between != and !==?
11. What are logical operators?
12. Explain &&.
13. Explain ||.
14. Explain !.
15. What is increment?
16. What is decrement?
17. Difference between prefix and postfix increment?
18. What is a ternary operator?

- 19. What is operator precedence?
 - 20. What is an expression?
 - 21. What happens in "10" + 5?
 - 22. What happens in "10" - 5?
 - 23. What is type coercion?
 - 24. Why is === generally preferred over == for strict comparison?
 - 25. What is the difference between an operator and an expression?
-

Important MCQs

1. Which operator gives the remainder?

- A. /
- B. %
- C. //
- D. **

Answer: B

2. What is the result?

10 % 3

- A. 3
- B. 0
- C. 1
- D. 10

Answer: C

3. Which operator checks strict equality?

- A. =
- B. ==
- C. ===
- D. !=

Answer: C

4. What is the output?

`console.log(5 + "5");`

- A. 10
- B. "10"
- C. "55"
- D. Error

Answer: C

5. Which operator represents logical AND?

- A. &
- B. &&
- C. ||
- D. !

Answer: B

6. Which operator represents logical OR?

- A. &&
- B. !
- C. ||
- D. or

Answer: C

7. Which operator represents logical NOT?

- A. !
- B. !=
- C. not
- D. ~

Answer: A

8. What is the output?

let x = 10;

console.log(x++);

- A. 9
- B. 10
- C. 11
- D. Error

Answer: B

9. What is the output?

let x = 10;

console.log(++x);

- A. 9
- B. 10
- C. 11
- D. Error

Answer: C

10. Which is the conditional/ternary operator?

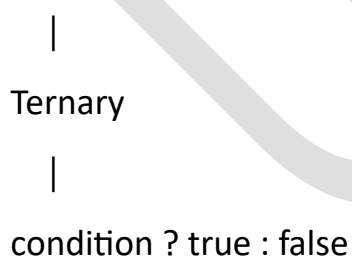
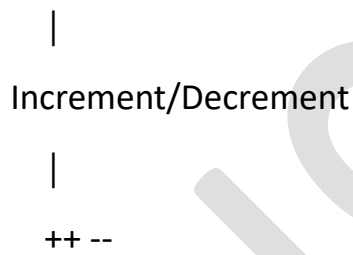
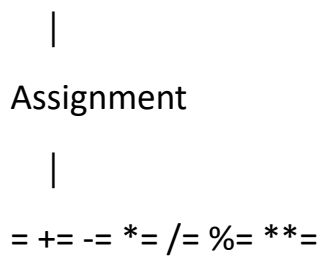
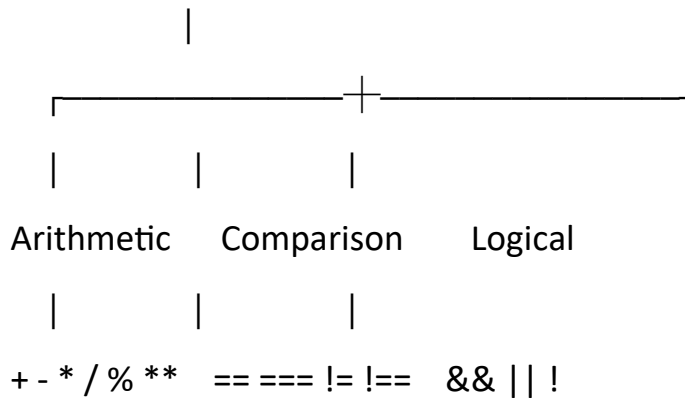
- A. ??
- B. ?:
- C. =>
- D. ::

Answer: B

Blackboard Summary

At the end of the lecture, students should have this quick-reference chart:

JAVASCRIPT OPERATORS



Most important rules to remember

- + → Addition / String concatenation
- → Subtraction
- * → Multiplication
- / → Division

% → Remainder

** → Power

= → Assignment

== → Loose equality

=== → Strict equality

&& → AND

|| → OR

! → NOT

++ → Increment

-- → Decrement

?: → Ternary